

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 4522 PROJE RAPORU

Ağ Tabanlı Paralel Dağıtım Sistemleri

Gökten Bayram

21290442

Veritabanı Yedekleme Ve Otomasyon Çalışması video linki

<https://drive.google.com/file/d/1opigBQAFAoxy1t6EretVx0Wf7ZWpJw6X/view?usp=sharing>

Veritabanı Güvenliği Ve Erişim Kontrolü Çalışması video linki

https://drive.google.com/file/d/16qrMhPj-7RwKUYMxrbVd74H_EFTWBk_x/view?usp=sharing

Veritabanı Performans Optimizasyonu Ve İzleme Çalışması video linki

https://drive.google.com/file/d/1DvK4L4otpmTkdRKhxYyVL7HJj82ai4Q_/view?usp=sharing

İÇİNDEKİLER

1. Veritabanı Güvenliği Ve Erişim Kontrolü Çalışması.....	3
1.1. Giriş ve Proje Özeti.....	3
1.2. Erişim Yönetimi ve Rol Tabanlı Yetkilendirme.....	3
1.3. Veri Şifreleme.....	4
1.4. SQL Injection Testleri Ve Korunması.....	5
1.5. Audit Logları Ve Kullanıcı Aktivitelerinin İzlenmesi.....	6
1.6. Sonuç.....	6
2. Veritabanı Performans Optimizasyonu Ve İzleme Çalışması.....	7
2.1. Giriş ve Senaryo Tasarımı.....	7
2.2. Veritabanı İzleme Ve Darboğaz Teşhisi.....	7
2.3. İndeks Yönetimi Ve Sorgu İyileştirme.....	8
2.4. Veri Yöneticisi Roller ve Erişim Yönetimi.....	9
2.5. Sonuç.....	10
3. Veritabanı Yedekleme Ve Otomasyon Çalışması.....	11
3.1. Giriş ve Sistem Mimarisi.....	11
3.2. Veritabanı Ve Hedef Tablo Yapısı.....	11
3.3. Otomasyon Script Geliştirilmesi.....	11
3.4. Cron Job İle Zamanlama Ve Otomasyon.....	12
3.5. Denetim, Raporlama Ve Doğrulama Çıktıları.....	13
3.6. Sonuç.....	13

1. VERİTABANI GÜVENLİĞİ VE ERİŞİM KONTROLÜ ÇALIŞMASI

1.1. GİRİŞ VE PROJE ÖZETİ

Bu rapor, veritabanı sistemlerinde veri güvenliğini sağlamak, yetkisiz erişimleri engellemek, veri bütünlüğünü korumak ve sistem aktivitelerini izlenebilir kılmak amacıyla gerçekleştirilen güvenlik konfigürasyonlarını içermektedir. Projede ana veritabanı olarak PostgreSQL ve veritabanı yönetim arayüzü olarak DBeaver kullanılmıştır. Bu doğrultuda, SQL Server üzerinde istenen tüm güvenlik katmanları (Authentication, TDE, SQL Injection Protection, Audit), PostgreSQL mimarisindeki birebir teknik karşılıkları araştırılarak başarılı bir şekilde uygulanmıştır.

1.2 ERİŞİM YÖNETİMİ VE ROL TABANLI YETKİLENDİRME

Veritabanı güvenliğinin ilk katmanı, sisteme kimlerin hangi haklarla erişebileceğini belirlemektir.. Kullanıcılara sistem üzerinde sadece ihtiyaç duydukları minimum yetkiler tanımlanmıştır.

1.2.1. Uygulanan SQL Konfigürasyonu

PostgreSQL üzerinde SQL Server Authentication mantığına karşılık olarak şifre korumalı ve login yetkisi olan kısıtlı bir deneme rolü tanımlanmıştır.

```
-- Kısıtlı bir kullanıcı (rol) oluşturulması
CREATE ROLE deneme_kullanici WITH LOGIN PASSWORD '123';

-- Kullanıcıya sadece ilgili şema üzerinde çalışma izni verilmesi
GRANT USAGE ON SCHEMA public TO deneme_kullanici;

-- Kullanıcının tablolara sadece okuma yetkisinin tanımlanması
GRANT SELECT ON ALL TABLES IN SCHEMA public TO deneme_kullanici;
```

1.2.2. Güvenlik Testi ve Doğrulama

Oluşturulan deneme_kullanici hesabı ile DBeaver üzerinde yeni bir oturum açılmış ve sisteme yetkisiz veri ekleme testi yapılmıştır:

```
-- Deneme oturumunda veri ekleme denemesi
INSERT INTO personel (ad, soyad) VALUES ('Zararlı', 'Deneme');
```

Test Sonucu: PostgreSQL erişim kontrol mekanizması sorgunun çalışmasını engellemiş ve DBeaver arayüzünde “SQL Error [42501]: ERROR: permission denied for table personel” hatası görülmüştür.

Bu testle birlikte, rol tabanlı erişim kontrolünün başarıyla yapılandırıldığı ve yetkisiz veri manipülasyonlarının önüne geçildiği kanıtlanmıştır.

1.3. VERİ ŞİFRELEME

SQL Server bünyesinde yer alan TDE teknolojisi, verilerin disk üzerinde şifreli olarak saklanmasını sağlar. PostgreSQL mimarisinde bu güvenlik katmanı, askeri düzeyde şifreleme algoritmalarını destekleyen yerleşik pgcrypto modülü kullanılarak sütun bazlı şifreleme kullanılmıştır.

1.3.1. Uygulanan SQL Konfigürasyonu

Personel maaşları gibi hassas verilerin sızdırılmasını önlemek amacıyla veriler düz metin yerine simetrik bir anahtar kelime kullanılarak şifrelenmiş ve BYTEA (binary) formatta saklanmıştır.

```
-- Şifreleme eklentisinin aktifleştirilmesi
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- Hassas verileri tutacak tablonun oluşturulması
CREATE TABLE hassas_personel_bilgileri (
    personel_id INT PRIMARY KEY,
    ad_soyad VARCHAR(100),
    maas_bilgisi BYTEA -- Şifreli veri alanı
);

-- Verinin AES tabanlı pgp_sym_encrypt fonksiyonu ve gizli anahtar ile eklenmesi
INSERT INTO hassas_personel_bilgileri (personel_id, ad_soyad, maas_bilgisi)
VALUES (1, 'Ahmet Yılmaz', pgp_sym_encrypt('45000 TL', 'gizli_sifre_123'));
```

1.3.2. Güvenlik Testi ve Doğrulama

Veritabanına dışarıdan yetkisiz bir erişim sağlandığında veya düz bir SELECT * sorgusu atıldığında verinin durumu incelenmiştir:

```
SELECT * FROM hassas_personel_bilgileri;
```

Test Sonucu: maas_bilgisi sütununda "45000 TL" yerine \xc30d04070302... şeklinde anlamsız bir binary blok görüntülenmiştir. Verinin aslı ancak doğru anahtara sahip yetkili uygulama tarafından aşağıdaki şekilde çözülebilmektedir:

```
-- Sadece gizli anahtarı bilenlerin veriyi çözmesi
SELECT ad_soyad, pgp_sym_decrypt(maas_bilgisi, 'gizli_sifre_123') AS cozulmus_maas FROM
hassas_personel_bilgileri;
```

Bu sayede, fiziksel veritabanı dosyaları (backup dosyaları dahil) çalınsa bile siber saldırganlar için veriler tamamen işlevsiz hale getirilmiştir.

1.4. SQL INJECTION TESTLERİ VE KORUNMASI

SQL Injection, web uygulamalarındaki girdi alanlarından sızdırılan zararlı SQL komutları vasıtasıyla veritabanı mantığının manipüle edilmesi saldırısıdır. Korumasız sistemlerde saldırganlar ' OR '1'='1 gibi mantıksal ifadelerle tüm veritabanı içeriğini sızdırabilirler. Projede bu zafiyeti tamamen kapatmak adına Prepared Statements (Önceden Derlenmiş Sorgular) mimarisi uygulanmıştır.

1.4.1. Uygulanan SQL Konfigürasyonu

Sorgunun iskeleti ve çalıştırılma planı veritabanı motoruna önceden öğretilmiş, kullanıcı girdileri ise sisteme sadece birer "parametre" olarak tünellenmiştir.

```
-- Test için örnek üye tablosunun oluşturulması
CREATE TABLE test_uyeler (
  uye_id SERIAL PRIMARY KEY,
  kullanıcı_adi VARCHAR(50),
  sifre VARCHAR(50)
);

INSERT INTO test_uyeler (kullanıcı_adi, sifre) VALUES ('admin', 'SuperSifre123');
INSERT INTO test_uyeler (kullanıcı_adi, sifre) VALUES ('ayse_dev', 'Ayse9988!!');

-- Prepared Statement (Önceden Derlenmiş Sorgu) Şablonu
PREPARE yeni_giriş_sorgusu (text) AS
  SELECT * FROM test_uyeler WHERE kullanıcı_adi = $1;
```

1.4.2. Güvenlik Atak Testi ve Savunma Doğrulaması

Sistem üzerinde hem normal bir istek hem de bir SQL Injection saldırısı simüle edilerek sonuçlar karşılaştırılmıştır:

```
-- Senaryo A: Normal Kullanıcı Girişi
EXECUTE yeni_giriş_sorgusu('admin');
-- Sonuç: 'admin' kullanıcısının satırı başarıyla listelenmiştir.

-- Senaryo B: SQL Injection Saldırı Girişimi
EXECUTE yeni_giriş_sorgusu('admin" OR "1"="1');
```

Saldırı Test Sonucu: Senaryo B çalıştırıldığında veritabanı boş sonuç (empty set) döndürmüştür. Sistem çökmemiş veya diğer kullanıcıların verilerini sızdırmamıştır.

Savunma Mekanizması Analizi: Prepared Statements kullanımı sayesinde, PostgreSQL girdi olarak gelen ' OR '1'='1 ifadesini bir SQL komutu olarak çalıştırmamış, bunu harfi harfine tek bir string parametresi (bir kullanıcı adı) olarak kabul etmiştir. Tabloda adı tam olarak bu zararlı kod olan bir üye bulunmadığı için sorgu güvenli bir şekilde boş dönmüş ve atak boşa çıkarılmıştır.

1.5. AUDIT LOGLARI VE KULLANICI AKTİVİTELERİNİN İZLENMESİ

Veritabanı güvenliğinin sürdürülebilir olması, sistemdeki tüm hareketlerin izlenebilir ve denetlenebilir olmasına bağlıdır. SQL Server Audit özelliğine karşılık olarak, PostgreSQL bünyesinde yer alan ve tüm canlı oturumları, sorguları, istemci IP adreslerini anlık olarak kaydeden pg_stat_activity sistem kataloğu kullanılmıştır.

1.5.1. Denetim Sorgusu

Sistem yöneticilerinin ve güvenlik operasyon merkezlerinin (SOC) geriye dönük adli analiz (forensics) yapabilmesi amacıyla aşağıdaki audit izleme sorgusu kurgulanmıştır:

```
SELECT
  pid AS islem_id,
  username AS baglanan_kullanici,
  client_addr AS istemci_ip_adresi,
  backend_start AS baglanti_baslangici,
  query AS calistirilan_son_sorgu,
  state AS durum
FROM pg_stat_activity
WHERE username IS NOT NULL;
```

1.5.2. İzleme Sonuçlarının Değerlendirilmesi

Bu denetim sorgusu çalıştırıldığında;

- Veritabanına o an hangi kullanıcıların bağlı olduğu,
- İsteklerin hangi IP adreslerinden (client_addr) geldiği,
- En önemlisi de kötü niyetli veya hatalı olabilecek en son çalıştırılan SQL sorgusunun ne olduğu (query) açıkça raporlanmıştır.

Bu altyapı sayesinde veritabanı üzerinde tam bir görünürlük sağlanmış, şüpheli aktivitelerin ve yetki aşımalarının anında tespit edilmesinin önü açılmıştır.

1.6. SONUÇ

Proje sonucunda;

1. Rol tabanlı erişim ile Yetkisiz Erişimler,
2. pgcrypto şifrelemesi ile Veri Hırsızlığı ve Sızıntıları,
3. Prepared Statements ile SQL injection engellenmiş,

Gerçekleştirilen tüm bu çalışmalar, veritabanı yönetim sistemlerinin sadece veri saklama aracı değil, aynı zamanda sıkı güvenlik politikalarıyla korunması gereken kritik birer altyapı bileşeni olduğunu ortaya koymaktadır.

2. VERİTABANI PERFORMANS OPTİMİZASYONU VE İZLEME ÇALIŞMASI

2.1. GİRİŞ VE SENARYO TASARIMI

Bu rapor, büyük veri hacmine sahip ilişkisel veritabanlarında meydana gelen performans darboğazlarının tespit edilmesi, sorgu maliyetlerinin izlenmesi ve indeksleme stratejileriyle sistem performansının optimize edilmesini içeren mimari çalışmaları kapsamaktadır.

Ödev dokümanında belirtilen SQL Server odaklı izleme (SQL Profiler, DMV) ve yönetim isterleri, projenin geliştirildiği PostgreSQL veritabanı motoru ve DBeaver arayüzü üzerinde birebir simüle edilmiştir. Performans farkını kararlı ve ölçeklenebilir şekilde gözlemleyebilmek adına, sistemde 5 Milyon satır rastgele veri içeren “satis_raporlari” tablosu üretilmiş ve ilgili işlemler bu veri seti üzerinde uygulanmıştır.

2.2. VERİTABANI İZLEME VE DARBOĞAZ TEŞHİSİ

SQL Server dünyasındaki SQL Profiler ve DMV (Dynamic Management Views) mekanizmalarının temel işlevi, çalışan bir sorgunun yürütme planını çıkararak disk, RAM ve CPU üzerindeki yükünü raporlamaktır. PostgreSQL mimarisinde bu izleme katmanı “EXPLAIN ANALYZE” komutu ile gerçekleştirilmiştir.

2.2.1. İndeksiz Durum İzleme Sorgusu

Sistem performansını test etmek ve indeksleme öncesindeki temel veriyi (baseline) elde etmek amacıyla, 5 Milyon satırdan oluşan satis_raporlari tablosunda belirli bir müşteri ID'sine göre arama yapılmış ve sorgunun yürütme planı EXPLAIN ANALYZE komutu ile izlenmiştir:

```
-- Belirli bir müşteri ID'sine göre 5 Milyon satırlık tabloda izleme yapılması
EXPLAIN ANALYZE
SELECT * FROM satis_raporlari WHERE musteri_id = 4956788;
```

2.2.2. İzleme Raporunun Satır Satır Analizi

Sorgu çalıştırıldığında elde edilen metinsel yürütme planı ve sistem çıktısı şu şekildedir:

- Gather (Workers Launched: 2): PostgreSQL planlayıcısı, veri setinin büyüklüğünü analiz ederek sorguyu paralelleştirmiştir. Ana işlemci çekirdeğine ek olarak 2 adet yardımcı arka plan işçisi ile birlikte 5 milyon satırlık yük 3 koldan bölüşülmüştür.
- Parallel Seq Scan on satis_raporlari: “musteri_id” sütunu üzerinde herhangi bir indeks bulunmadığı için, devreye giren işçiler doğrudan indeks tablosuna yönlenebilmiştir. Bunun yerine tablonun disk üzerindeki başlangıç noktasından bitiş noktasına kadar tüm satırları sırayla tek tek okumak zorun kalmıştır. Bu kritik bir darboğaz olduğunun göstergesidir.

- Rows Removed by Filter: Tablonun 3 koldan taranması neticesinde, işçi başına tam olarak 1.666.667 satır düşmüştür. İşçi, kendisine ayrılan bu bloğu baştan sona taramış, aranan ID dışındaki tüm satırları filtreleyerek çöpe atmıştır. İndeks olmadığı için, aranan veri ilk satırlarda bulunsa dahi, ileride başka bir kayıt olup olmadığını kontrol etmek adına tüm blok zorunlu olarak taranmıştır. Bu yüzden sorgulanan ID değişse bile elenen satır sayısı matematiksel olarak bu blok boyutunda sabit kalmaktadır.
- Execution Time: Sorgunun tamamlanması yaklaşık olarak 98.238 milisaniye sürmüştür. Kurumsal ve canlı ölçekteki bir veritabanı mimarisinde, tek bir satıra ulaşmak için harcanan bu süre kabul edilemeyecek düzeyde yüksek bir gecikmedir ve sistem kaynaklarının israf edildiğini göstermektedir.

2.3. İNDEKS YÖNETİMİ VE SORGU İYİLEŞTİRME

Tarama maliyetini tamamen ortadan kaldırmak, disk giriş/çıkış işlemlerini minimize etmek ve sorguyu optimize etmek amacıyla veritabanı üzerinde İndeks Yönetimi katmanı devreye alınmıştır.

2.3.1. Uygulanan Mimari Çözüm (B-Tree İndeksi)

Sorgularda filtreleme amacıyla yoğun olarak kullanılan “musteri_id” sütunu üzerine, arama karmaşıklığını doğrusal seviyeden logaritmik seviyeye düşüren bir B-Tree indeksi inşa edilmiştir. Bu sayede arama maliyeti matematiksel olarak $O(n)$ seviyesinden $O(\log n)$ seviyesine indirilmiştir.

```
-- Doğru sütun üzerinde indeks oluşturulması
CREATE INDEX idx_satis_musteri_id ON satis_raporlari(musteri_id);
```

2.3.2. Optimizasyon Sonrası İzleme ve Doğrulama

İndeks oluşturulduktan sonra aynı sorgu EXPLAIN ANALYZE ile yeniden izlenmiştir:

```
-- İndeks sonrası performans durumunun yeniden izlenmesi
EXPLAIN ANALYZE
SELECT * FROM satis_raporlari WHERE musteri_id = 4956788;
```

Performans Karşılaştırma Raporu:

- Index Scan Dönüşümü: Daha önce CPU ve disk maliyetine yol açan 'Parallel Seq Scan' yapısı tamamen terk edilmiş; sistem doğrudan oluşturulan indekse giderek verinin adresini nokta atışı tespit etmiştir (Index Scan).
- Maliyet Sıfırlama: Rows Removed by Filter değeri tamamen sıfıra inmiştir. Sistem artık milyonlarca satırı çöpe atmamaktadır.
- Süre Optimizasyonu: Sorgu süresi 98.238 milisaniyeden 0.183 milisaniyeye düşmüştür. Sistem yaklaşık 537 kat hızlandırılarak sorgu iyileştirme adımı başarıyla tamamlanmıştır.

2.4. VERİ YÖNETİCİSİ ROLLERİ VE ERİŞİM YÖNETİMİ

Büyük ölçekli sistemlerde, siber güvenlikteki "Görevler Ayrılığı" prensibi kritik bir rol oynar. Veritabanı performansını izlemek, yavaş sorguları teşhis etmek ve indeks yönetimi yapmakla görevli olan "Performans Analistlerinin", tablo içeriklerine doğrudan müdahale etme, silme veya değiştirme yetkilerinin olması bir güvenlik açığıdır.

2.4.1. Uygulanan SQL Konfigürasyonu

PostgreSQL üzerinde, sadece sistem sağlığını ve performans DMV'lerini izlemekle görevli kısıtlı bir veri yöneticisi rolü tanımlanmıştır.

```
-- Sadece performans yönetiminden sorumlu analist rolü  
CREATE ROLE dba_analist WITH LOGIN PASSWORD 'Analist123!';  
  
-- SQL Server DMV tablolarına karşılık gelen pg_monitor yetki paketinin atanması  
GRANT pg_monitor TO dba_analist;
```

2.4.2. Yetki Sınırlandırma ve Güvenlik Doğrulaması

DBeaver üzerinde dba_analist oturumu açılarak yapılan iki yönlü canlı test sonuçları aşağıdadır:

Olay A (İzleme Yetkisi): Analist, sistemin genel sorgu havuzunu denetlemek için izleme sorgusunu çalıştırabilmektedir:

```
SELECT pid, username, query, state FROM pg_stat_activity;
```

Sonuç: Başarılı. Sistem anlık logları analiste sunmaktadır.

Olay B (Veri Müdahale Kısıtı): Aynı analist, veritabanından veri silmeye veya değiştirmeye çalıştığında:

```
DELETE FROM satis_raporlari WHERE rapor_id = 1;
```

Sonuç: Engellendi. Veritabanı motoru anında şu güvenlik hatasını döndürmüştür: SQL Error [42501]: ERROR: permission denied for table satis_raporlari

Bu sayede, performans yönetim süreçlerinin veri gizliliği ve bütünlüğünü ihlal etmeden güvenli bir izole alanda yürütülmesi garanti altına alınmıştır.

2.5. SONUÇ

Gerçekleştirilen bu proje çalışmasında, 5 Milyon satırlık büyük veri hacmi altında çalışan bir sistemin performans dinamikleri analiz edilmiştir.

- 1 . EXPLAIN ANALYZE mekanizması ile Sorgu İzleme yapılmış ve amelelik oluşturan Parallel Seq Scan hatası saptanmıştır.
- 2 . CREATE INDEX adımıyla İndeks Yönetimi yürütülmüş, sorgu yanıt süresi 537 kat düşürülerek donanım kaynaklarının gereksiz tüketimi önlenmiştir.
- 3 . pg_monitor ve kısıtlı rol yapısıyla veri yöneticisi rolleri yapılandırılarak, kurumsal siber güvenlik standartlarına uygun bir izleme altyapısı kurulmuştur.

Yapılan optimizasyonlar, veritabanlarında kod kalitesi ve doğru mimari kurgunun, donanım yükseltmelerine ihtiyaç duymaksızın sistemleri ne kadar büyük ölçüde hızlandırabileceğini somut olarak ortaya koymuştur.

3. VERİTABANI YEDEKLEME VE OTOMASYON ÇALIŞMASI

3.1. GİRİŞ VE SİSTEM MİMARİSİ

Bu çalışma kapsamında, bir veritabanının yönetim süreçlerini optimize etmek, veri güvenliğini sağlamak, insan hatasını minimuma indirmek ve düzenli denetim/raporlama mekanizmasını devreye almak amacıyla uçtan uca bir Yedekleme Otomasyon Sistemi kurulmuştur. SQL Server Agent Yerine: Unix tabanlı işletim sistemlerinin yerleşik zamanlanmış görev servisi olan Cron (Crontab) kullanılmıştır. PowerShell / T-SQL Yerine: Dağıtım ve otomasyon süreçlerinin standardı olan Bash Scripting dili tercih edilmiştir. Yedekleme Motoru: PostgreSQL'in güçlü ve esnek yerel yedekleme aracı olan pg_dump komut seti kullanılmıştır.

3.2. VERİTABANI VE HEDEF TABLO YAPISI

Çalışmada hedef olarak belirlenen ana veritabanı yapısı DBeaver arayüzü üzerinde modellenmiştir. Sistem yükünü optimize etmek amacıyla tüm veritabanı yerine spesifik bir test tablosu hedef alınmıştır.

- Veritabanı Adı (DB_NAME): postgres
- Şema (Schema): public
- Hedef Tablo (Table): test_uyeler

3.3. OTOMASYON SCRIPT'İNİN GELİŞTİRİLMESİ

Süreci otomatikleştiren Bash script'i, modüler ve hata toleranslı bir yapıda tasarlanmıştır. Kod bloğu ve satır satır teknik açıklamaları aşağıda yer almaktadır:

```
#!/bin/bash
```

```
# 1. DEĞİŞKEN TANIMLAMALARI
```

```
DB_NAME="postgres"
```

```
BACKUP_DIR="/Users/gktn/Desktop/BACKUP_DIR"
```

```
DATE=$(date +%Y-%m-%d_%H-%M-%S)
```

```
BACKUP_FILE="$BACKUP_DIR/${DB_NAME}_backup_${DATE}.sql"
```

```
LOG_FILE="$BACKUP_DIR/yedekleme_raporu.log"
```

```
# 2. DİZİN OLUŞTURMA VE LOG BAŞLANGICI
```

```
mkdir -p "$BACKUP_DIR"
```

```
echo "--- Yedekleme İşlemi Başladı: $(date) ---" >> "$LOG_FILE"
```

3. GÜVENLİK KATMANI (ŞİFRE YÖNETİMİ)

```
export PGPASSFILE="$HOME/.pgpass"
```

4. ÇEKİRDEK YEDEKLEME KOMUTU

```
/Library/PostgreSQL/18/bin/pg_dump -h localhost -U postgres -d "$DB_NAME" -t public.test_uyeler -F p -f "$BACKUP_FILE" 2>> "$LOG_FILE"
```

5. İŞLEM KONTROLÜ, RAPORLAMA VE OTOMATİK UYARI

```
if [ $? -eq 0 ]; then
```

```
    echo "[BAŞARILI] $DATE tarihinde yedek başarıyla alındı. Dosya: $BACKUP_FILE" >> "$LOG_FILE"
```

```
    echo "RAPOR: Yedekleme boyutu: $(du -sh $BACKUP_FILE | cut -f1)" >> "$LOG_FILE"
```

```
else
```

```
    echo "[HATA] $DATE tarihindeki yedekleme BAŞARISIZ oldu!" >> "$LOG_FILE"
```

```
    # macOS Ekran Bildirimi Uyarı Sistemi
```

```
    osascript -e "display notification 'Veritabanı yedekleme işlemi BAŞARISIZ oldu!' with title 'YEDEKLEME HATASI'
```

```
    sound name 'Basso'"
```

```
fi
```

```
echo "--- İşlem Bitti: $(date) ---" >> "$LOG_FILE"
```

```
echo "" >> "$LOG_FILE"
```

Kodun Teknik Analizi:

1. Shebang (#!/bin/bash): Dosyanın Bash kabuğu tarafından yorumlanacağını garanti eder.
2. Dinamik Adlandırma (DATE): Zaman damgası kullanılarak üretilen yedeklerin benzersiz olması ve geçmişe dönük izlenebilirliği sağlanmıştır.
3. Güvenlik ve Kimlik Doğrulama (export PGPASSFILE): Şifrenin script içine açık metin (cleartext) olarak yazılması engellenmiştir. Kullanıcı dizinindeki gizli ve chmod 0600 ile kısıtlanmış .pgpass dosyası çevre değişkeni olarak sisteme öğretilmiştir.
4. Hata Yönlendirmesi (2>> "\$LOG_FILE"): pg_dump komutu esnasında oluşabilecek standart hata çıktıları doğrudan rapor loguna yönlendirilerek denetim kolaylaştırılmıştır.
5. Durum Kontrolü (\$?): Bir önceki komutun çıkış kodu incelenir. 0 ise başarılı sayılır.

3.4. CRON JOB İLE ZAMANLAMA VE OTOMASYON

Yazılan scriptin insan müdahalesi olmadan çalışması için macOS üzerindeki yerleşik zamanlanmış görev yöneticisi (Crontab) kullanılmıştır. Terminal üzerinden crontab -e komutu çalıştırılarak sisteme aşağıdaki kural eklenmiştir:

```
* * * * * /Users/gktn/Desktop/yedeklee.sh
```

Not: Üretim (Production) ortamlarında bu görevin her gece yarısı 0 0 * * * periyodunda çalışması optimize edilirken, bu çalışmanın canlı testi ve doğrulanması amacıyla zaman parametresi "her dakika başı" (* * * * *) olarak set edilmiştir.

3.5. DENETİM, RAPORLAMA VE DOĞRULAMA ÇIKTILARI

Sistemin çalışması esnasında üretilen loglar ve fiziksel yedek dosyaları incelenerek otomasyonun başarısı kanıtlanmıştır.

A. Raporlama ve Süreç Log Çıktısı (yedekleme_raporu.log):

Sistemin ürettiği denetim raporu dosyasının içeriği şu şekildedir:

```
--- Yedekleme İşlemi Başladı: Fri May 29 01:47:00 +03 2026 ---  
[BAŞARILI] 2026-05-29_01-47-00 tarihinde yedek başarıyla alındı. Dosya: /Users/gktn/Desktop/BACKUP_DIR/  
postgres_backup_2026-05-29_01-47-00.sql  
RAPOR: Yedekleme boyutu: 4.0K  
--- İşlem Bitti: Fri May 29 01:47:00 +03 2026 ---
```

3.6. SONUÇ

Bu çalışma ile bir veritabanı yöneticisinin yapması gereken rutin yedekleme operasyonları tamamen otomatik hale getirilmiştir. Sistem; zamanlanmış görev mimarisi, güvenli kimlik doğrulama tüneli, detaylı süreç denetim loglaması ve dinamik hata bildirim mekanizması ile endüstriyel standartlara uygun, güvenli ve optimize bir yapıya kavuşturulmuştur.